

大模型应用八股文总结

RAG

1. RAG是什么？

RAG是检索增强生成，LLM的回答一般基于已训练的知识库，训练的知识库有限，容易产生幻觉问题，因此可以通过RAG技术，额外引入外部的知识库，在查询时检索向量数据库找到相关的文档一起加入到提示词中提供给模型，生成更准确的回复。RAG一般包括几个重要的部分：知识库的准备，文档切分，嵌入模型，向量数据库，查询检索，生成回答。

2. Query查询优化的方法有哪些？

用户写的query可能出现偏口语化不规范的情况，而向量数据库中保存的都是偏专业化的信息，如果直接用原始查询去检索可能不能得到精准的答案，因此需要进行query重写。

常见的优化方法有以下：

- 将复杂问题拆解成多个子问题，可以逐步依赖，也可以独立求解后合并答案。
- 先用大模型生成一个假设性的答案文档，再将其作为查询向量去检索，从而提升长尾或专业问题的召回。
- 把具体问题提升为更抽象的问题，方便召回更广泛的相关文档。
- 把用户的原始问题从多个角度重写（同义替换、语义扩展、纠错、标准化），分别检索，再合并结果，提升召回全面性。
- 把自然语言 query 转换成结构化搜索参数，通过 NER、关键词抽取等技术，把 query 转成更标准的检索条件。

3. RAG中为什么要做分词？分词切片有哪些方式？

原本数据文档内容较多，不做分词则会对检索造成较大的压力，并且检索出来的信息不够精准，我只需要其中一部分的信息但直接检索出来整篇文档肯定是不合适的，合理的分词方式可以使得检索效率更高并且内容更精准。

固定大小切片 (Fixed-size Chunking)：最简单，按字符数硬切。**缺点**：极易切断句子，语义损失严重。

递归字符切片 (Recursive Character Splitting)：设置一组层级分隔符（如 `\n\n`，`\n`，`。`，`，`）。**优点**：能尽量保持段落和句子的完整性，是目前的通用首选。

语义切片 (Semantic Chunking)：利用 Embedding 模型计算相邻句子间的相似度，当相似度低于阈值时断开。**优点**：完全基于语义。**缺点**：计算开销大，推理速度慢。

文档结构切片 (Document-Specific Splitting)：针对 Markdown、HTML 或有固定层级的 PDF。根据标题（H1-H3）或表格结构进行物理分割。

基于大模型的分块：利用大模型识别最优的分割点，从而实现分块。

4. 向量数据库在RAG中的作用？为什么你选用chroma？

向量数据库在RAG中主要是起到存储外部知识库的作用，将外部文档分片后，按照给定的embedding模型进行向量化，在用户查询后，将用户的查询也会向量化，然后从知识库中根据语义相似度检索出相关性较高的参考资料，然后一同发送给大模型，chroma数据库由两大核心组件，一个是HNSW索引，这是一个多层图结构，每一层的节点都会有相似度较高的节点形成边构成邻居关系，这是后续检索的核心，能够将时间复杂度从 $O(N)$ 降低为 $O(\log N)$ 还有一个是SQLite，用于存储原来文本数据，同时chroma也支持持久化。

chroma数据库检索的底层原理：首先将查询也向量化，然后到HNSW索引中去找，在每一层的邻居中找到相似度最高的节点，然后下钻直到最底层，在最底层维护一个大小为k的优先队列，k是检索时的参数，依次和最底层的邻居节点比较，找到k个相似度最高的节点，将id存入列表，最后拿着id去SQLite中找到对应的文本数据并返回。

5. 了解BM25算法吗？原理是什么？

BM25是为了解决TF-IDF算法存在的问题，TF是词频即词出现的频率，IDF是逆文档频率，用来描述词的重要性比如一个词每篇文档都有那么说明这个词不重要权重应该降低，采用这种方式会有天然的缺陷就是词频越高分数越高，但其实出现1次和出现10次不应该差10倍，长文档有天然优势，因为词越多命中概率就高，但其实长并不代表相关性高，因此出现了BM25算法来解决这些问题。具体实现是需要控制TF的不无限增长，出现次数越多后面的影响要逐渐趋于饱和，以及对文档做归一化，如果文档比平均文档长分母就变大，分数就会下降，解决长文档优势问题。BM25是基于词的精确匹配，来计算查询和文档的相关性，并不是基于语义的相似度。

6. RAG中为什么需要Rerank？

在检索阶段确实可以利用embedding模型和语义相似度的计算，得到有相关性的一系列文档，但是不一定足够精确，即最相关的文档的不一定是排在最前面的，因此需要引入rerank再对这部分文档做一个精排，使用更复杂的模型对文档进行深度排序，以提高结果的精确度。

具体的原理是rerank模型采用cross-encoder架构，将查询和候选段落一同作为模型的输入，模型输出的是一个相关性分数，然后根据这个相关性分数就可以实现排序。

在RAG项目中一般需要embedding和rerank结合起来使用，因为rerank模型计算量太大，因此不适合直接对所有文档进行rerank排序，相比之下embedding模型的效率会高很多，因此可以先用embedding筛选出一部分具有相关性的文档，再用rerank模型进行按照相关性的重排序得到精确的文档。

7. 在你的系统中RAG检索出来质量很低你觉得有哪些原因？以及该怎么解决呢？

首先是切片层面，切片策略选取不当，导致检索到的文档是残缺的，自然会影响最终的RAG质量，需要采用语义切片或者父子文档切片。其次是embedding层面，模型能力不足也会导致RAG效果一般，可以采用混合检索策略结合BM25关键词的搜索，弥补语义检索的缺陷，对embedding模型进行专门的微调。用户查询太简单，偏向于口语化，需要进行query的重写。没有rerank，将检索的top-k结果直接给大模型没有再精细排序，也会导致质量低。

8. 如何设计RAG的评估体系？

要围绕三个方面综合评估，第一就是检索，检索到的内容是否相关，对用户的回答是否有帮助，检索端需要召回率和精确度两个指标，评估正确文档是否出现在检索结果中，以及在每个检索中出现的排名是否正确。第二就是生成，评估大模型生成的内容是否是基于检索到的文档生成的，还是胡编乱造的幻觉输出，这边指标就是忠实度或者幻觉率，第三就是端到端的体验，大模型的回答是否正确解决了用户的问题，是否答非所问，这边的指标是任务完成效果，这个一般需要和专家标注的答案对比。

Agent

1. function call

Function Call (函数调用): 模型接收到用户的提问之后通过推理，决定在当前上下文下是否需要调用外部工具，如果需要调用就输出符合规范的 JSON Schema格式的数据，LLM并无具体权限去执行代码，而是将函数的名称，参数等信息以json格式返回给agent，告诉agent该怎么去调用工具。

Function call是基于模型的微调实现的，在微调模型时就给模型喂了大量的数据集，包含用户的问题和一组JSON格式的函数描述，目标输出就是一段JSON Schema格式的数据，还引入了一些特殊标记，让模型学习在看到特殊语境时就能触发这些token，这样就可以让模型具备识别用户意图判断是否需要调用函数，不需要就直接正常返回，需要则根据微调时的记忆输出JSON格式数据。除了微调外，还可以用强化学习进一步增强模型function call的能力，让模型能够实现精准判断是否调用，不捏造参数，如果用户提供信息不足要学会追问而不是瞎回答（幻觉问题）。

2. mcp

MCP (Model Context Protocol): 模型上下文协议，专门用来规范agent和tool服务间是怎么交互的，定义了一套标准化的工具发现、调用和管理机制，如果没有MCP，agent如果需要使用外部世界的工具，由于不同工具返回的结果格式都是不一样的，不同的模型和框架所需要的数据格式也是不一样的，于是就会面临双重适配的问题，需要写大量不同的代码去让模型和工具适配，于是就有了MCP，它就像一个AI的USB接口，用统一的规则让不管哪个模型和哪个工具都可以直接交互，对于工具开发者来说，只需要按照MCP的规范写一遍代码，不管谁来调都返回一样的标准格式数据，对于Agent开发者也只需要实现一个MCP Client的接口，所有的工具都可以直接使用。

MCP协议有几个明显优势：MCP协议支持动态工具发现，可以在运行时添加新工具，大大提高了系统的灵活性。MCP协议提供了统一的工具接口规范，确保工具调用的规范性。MCP协议支持跨应用的工具共享，一个MCP服务器可以被多个客户端使用，提高了工具的复用性。

MCP具体实现：首先有两个端分别是MCP Client 和MCP Server，注册好MCP Server相关信息后，Client就会和Server建立连接，首先进行初始化initialize，其实就是个打招呼，有点像tcp连接的三次握手，然后是比较关键的tools/list，Client向Server请求获取工具列表，Server会返回所有可用工具的名称、参数列表、工具描述等信息给Client，这些都是在注册的时候就完成的。在后面的工具调用时，Client会再次和Server进行交互，请求参数method变为tools/call，client发送请求调用的工具及对应的参数，server则返回工具调用的结果信息。

MCP的三种通信方式：stdio是最简单的一种通信方式，不依赖于网络，只能在本地通过标准输入输出来进行交互，优点就是低延迟，本地交互肯定要比网络传输快得多，而且实现起来简单，缺点就是无

法满足远程交互的需求，并且不能同时处理多个请求；sse是基于http长连接实现的一个单向数据传输，由服务器主动向客户端推送事件，适用于实时数据更新的场景，比如股票行情，新闻动态推送等等，优点就是建立一次连接就可以一直推送数据，不需要客户端频繁发起请求，能够降低客户端和服务端之间的交互压力，并且可以远程访问，支持分布式部署，缺点就是只能单向传输，限制了双向交互，并且需要一直建立连接，如果有大量的客户端连接会造成服务器压力大；streamable http同样也是基于http实现的一个双向流式传输，允许数据以流的形式在客户端和服务端之间传输，适用于视频音频等大文件的传输，优点就是以流的形式分段传输，可以实现数据边传输边处理，避免等待整个文件全部加载完成才可以处理，缺点就是相对来说实现复杂。

3. skills

skills：试想一个场景，如果需要完成一系列流程，并且每个流程还有一些特殊规范，那么就需要将用户的问题，所有工具还有各种要求作为一个复杂的提示词一起发过去，那样非常消耗token，而且对用户来说太麻烦了，这个时候skill就可以解决这样的问题，skill表示agent具备某种能力，会规定如何按照一个规定的流程去完成某个任务，就像是一个专业的技能包，每个包下面会包含skill.md，参考资料和执行脚本，核心是skill.md文件，告诉agent何时使用这个skill以及具体的执行流程。skill采用接近式披露，agent启动时就会加载所有skill的元数据信息并注入进系统提示词里，也就是所有skill的名称和描述，这样大模型就可以知道所有Skill的能力，在真正处理用户问题的时候，大模型告诉agent选择具体的skill后agent时才会去加载对应的skill.md正文内容，然后再加入到上下文中发送给大模型，剩下的参考资料，执行脚本这些都是按需动态加载，当真正需要使用到的时候才会加载进去，可以更好更高效的利用上下文节省token。

MCP解决的是如何和外部交互，也就是Agent去哪拿数据的问题，Skill是教会Agent如何正确执行任务的问题，教Agent按照一个特定步骤、格式和标准完成任务。

4. tool是什么？

大模型本质上只是一个问答机器人，用户给出提问，它基于训练的数据进行回答，它没有办法与外部世界进行交互，这个时候就需要用到Tool了，Tool本质上也是函数，大模型可以利用Tool来获取外部世界的信息，具体流程是用户先将问题发送给agent，agent会将用户问题和工具列表一同发送给大模型，大模型就知道使用哪个工具去获取相关信息，但是大模型无法自己调用工具，它只能返回调用工具的指令，里面包含具体工具的名称和对应的参数，这就是tool calling，agent才会去真正执行调用工具的步骤，然后将工具调用的结果再发送给大模型，大模型根据结果输出对应的回答发送给agent，agent最后再返回给用户。

5. 了解coT吗？

Chain-of-Thought (CoT) 是一种改进的 Prompt 技术，目的在于提升大模型 LLMs 在复杂推理任务上的表现，对于复杂问题尤其是复杂的数学题大模型很难直接给出正确答案。COT 通过要求模型在输出最终答案之前，显式输出中间逐步的推理步骤这一方法来增强大模型的算数、常识和推理能力，简单但有效。zero-shot-coT是只添加一句提示词让模型一步步推理，而few-shot-coT则是给模型示例，详细描述问题-推理过程-答案，从而让模型具有推理新问题的能力。

6. agent是什么？

Agent是智能体，可以理解为一个“大预言模型+规划+记忆+工具使用”。规划指的是将复杂的任务拆解成可执行的子任务，并规划执行任务的流程，并且还可以在执行的过程中进行思考和反思，决定是否继续执行还是改进策略。agent开始处理用户的输入时，需要利用LLM先解析任务要求，然后将大任务拆解成多个子任务，然后LLM会阅读agent提供的tool描述，选择合适的工具然后返回执行计划给agent，也就是所谓的function calling，然后agent根据LLM返回的工具执行计划，用excutor去调用工具执行代码，最后还会将调用工具后的结果返回给LLM，由LLM将这些结果进行汇总总结之后返回给用户。

7. 为什么要用多智能体架构？和单智能体相比优缺点在哪里？

因为单个智能体等于一个人做所有的工作，多智能体等于团队协作模式，只用单智能体，当任务指令比较复杂的时候，容易无法完成指定任务，并且工具数量太多，也会降低任务的准确率，多智能体架构下，智能体专业化，将专业的事情交给专业的人来做，也不需要存储过多的上下文和记忆，往往效果会更好。

单智能体优点：实现简单，响应快，没有多智能体之间的通信开销，成本低，不需要为了协调和通信消耗额外的token，缺点：功能越多越容易产生幻觉，难以处理长链路复杂问题，所有工具描述和中间过程都挤在一个上下文窗口里，容易丢失重点。

多智能体优点：每个智能体专注于一个领域，任务完成度高，方便模块化扩展，增加新的智能体即可，状态隔离，例如解析智能体在解析过程中产生的垃圾信息会留在内部，传给主智能体的是干净的结构化结果。缺点：高昂的token成本，延迟增加，执行速度变慢，管理也会更加复杂

8. agent如何进行规划的？

规划这一步可以分解为两个方面，第一个是任务的分解，第二个是自我的反思。

底层的推理机制有基于coT，cot-sc，ToT，GoT，除了这些外，还需要有一些上层的框架来负责思考和行动的组织，比如React，Plan-and-Execute，Reflexion，Multi-Agent Collaboration等多种方法。

- 基于coT思维链的方式通过显示的逐步推理，让模型在生成答案前展开推理步骤。
- cotsc就是生成多个思维链，从中选择一个效果最好的。
- ToT是思维树，对于每一步生成多种可行的计划方案，这样会形成一个树结构，评估每个方向的可行性，筛选最优分支继续扩展，发现死胡同立刻回溯，遍历完所有节点生成一个最优的执行方案。
- GoT是思维图，可以看作是思维树的扩展，思维树只有分支，而思维图可以合并和回环，最终构成一个有向图结构，可以形成并行分解任务以及合并任务结果以及回环修正任务的复杂功能

而在我的项目中使用的是react框架，决策过程是一个不断思考、行动、观察的闭环，比如我提出了简历评估的任务，agent会先思考简历文件在哪，然后去执行简历读取，成功根据我给的路径读取到简历，然后会再思考需要解析简历，去调用简历解析工具，成功解析简历不断执行下去，直到最终完成简历评估的任务。

9. 如何实现React模式的Agent？

主要是依赖于一个系统提示词，在系统提示词里面可以规定好，让模型必须按照思考-行动-观察这样的流程去执行，并且给出每一步需要生成的格式是什么，然后在下面给出一些具体的例子，还要给出可用工具列表和一些注意事项，这其实就是核心原理。

具体代码实现是将系统提示词和用户的查询一起提供给模型，循环请求调用模型，首先获取输出中 `thought` 的部分，然后再检测是否有 `final_answer`，如果有说明已经完成最终目标直接，如果没有则说明还没结束，解析 `action` 部分，提取出需要调用的工具名和参数列表，然后去执行对应的工具函数，将执行结果放到 `observation` 里面，并将其加入到 `message` 里，接着进行第二次循环，用上一步的输出结果作为第二次循环的调用参数。

ReAct 的缺点：第一，交互带来的开销增加，每一次工具的调用都要经过思考-行动-观察三步，在这中间又设计多次和大模型的交互，这些都会占据模型的上下文，耗时并且消耗 token。第二，虽然在观察中有纠错机制，但是过长的推理链当中，小的错误可能会被逐步累加，最终生成的答案偏离最优解。第三，它对于提示词和工具的描述设计要求高，因为工具的调用完全取决于人定义的工具描述，如果设计不清晰会导致模型无法准确分辨工具导致任务失败。

10. Plan and excute模式的Agent是怎么实现的？

主要包含几个部分，一个是制定计划的模型，还需要一个执行计划的Agent或者是任意工具。首先需要将用户问题给制定计划的模型，让他给出具体的执行计划，然后让执行Agent从第一步计划开始执行，直到执行计划所有步骤全部都完成了，可以将结果返回给用户。最初 `plan and excute` 模式的并没有内置反思机制，因此很容易出现中间某一步如果执行出错了就会出问题，这也是这个模式相比 `react` 的一个缺陷。如果需要改进的话那就是要在执行每一步计划完需要验证一下执行结果并且进行重新制定计划，这样才可以保证不会出错。

11. Reflexion框架是如何实现的？

依赖于四个核心组件，执行者，评估者和反思模块和记忆模块，执行者就是基于当前状态生成动作，评估者则对执行者生成的结果进行评估打分和理由，反思模块会结合前面的结果进行反思，生成改进建议，并存入记忆中，供下一次执行者使用，下一次执行者就会参考之前的反思结果进行优化决策，这是一个迭代执行的过程，直到最终Actor生成的结果符合要求。就比如可以用于代码生成，一开始执行者给出的代码很可能会包含错误，然后评估者就可以进行评估发现错误，反思模块就会结合评估的分数和原因给出指导性的修改建议，并且存入长期记忆中，这样执行者下一次生成代码就可以遵循意见进行修改，从而保证最终能生成一份正确的代码。

12. 如何让agent具备记忆功能，尤其是长期记忆？

短期记忆可以通过上下文窗口实现，将历史会话会塞进 `prompt` 中传递给模型，实现短期记忆，但是上下文窗口有限。短期记忆存储会话中产生的各类消息，包括用户输入、模型回复、工具调用及其结果等。这些消息直接参与模型推理，实时更新，并受模型的 `maxToken` 限制。当消息累积导致上下文窗口超出限制时，需要通过上下文工程策略（压缩、卸载、摘要等）进行处理，这也是上下文工程主要处理的部分。

长期记忆主要是基于外部存储：可以基于向量数据库和RAG的方式，将历史会话转为向量存入向量数据库，然后通过RAG的方式检索出历史会话内容。也可以历史会话转为结构化数据后存入文件或者数据库中，后续调用的时候都会先去外部存储中加载出历史会话数据。不同产品的长期记忆存储的内容可能有区别，比如可以存储用户画像的一些信息，或者是业务相关的一些经验知识。

长短期记忆的配合使用：

Record（写入）：从短期记忆的会话消息中提取有效信息，通过LLM进行语义理解和抽取，存储到长期记忆中

Retrieve（检索）：根据当前用户查询，从长期记忆中检索相关信息，注入到短期记忆中作为上下文，辅助模型推理

13. 维护上下文的关键问题是什么？如何解决短期记忆因为上下文窗口带来的问题？

关键问题一个是会产生大量的垃圾信息，将这些上下文全部丢给LLM，容易使得注意力迷失，第二个就是token消耗，上下文越长，单次推理消耗的token更多。

上下文缩减（Context Reduction）

上下文缩减通过减少上下文中的信息量来降低 token 消耗，主要有两种方法：

保留预览内容：对于大块内容，只保留前 N 个字符或关键片段作为预览，原始完整内容被移除

总结摘要：使用 LLM 对整段内容进行总结摘要，保留关键信息，丢弃细节

这两种方法都会导致信息丢失，但能有效减少 token 消耗。

上下文卸载（Context Offloading）

上下文卸载主要解决被缩减的内容是否可恢复的问题。当内容被缩减后，原始完整内容被卸载到外部存储（如文件系统、数据库等），消息中只保留最小必要的引用（如文件路径、UUID 等）。当需要完整内容时，可以通过引用重新加载。

优势：上下文更干净，占用更小，信息不丢，随取随用。适用于网页搜索结果、超长工具输出、临时计划等占 token 较多的内容。

上下文隔离（Context Isolation）

通过多智能体架构，将上下文拆分到不同的子智能体中（类似单体拆分成多个微服务）。主智能体编写任务指令，发送给子智能体，子智能体的整个上下文仅由该指令组成。子智能体完成任务后返回结果，主智能体不关心子智能体如何执行，只需要结果。

14. 什么是A2A协议？

A2A协议叫做Agent to Agent协议，顾名思义就是会发生在多个Agent之间进行交互时需要用到的一个协议，主要在两个环节需要用到，一个是注册Server Agent的时候，向Server Agent发送请求，然后Server Agent会返回一个Agent card，还有就是在Client Agent 和Server Agent交互的时候会用到，Client Agent会向Server Agent发送请求，请求中会包含Message，然后Server Agent自己会进行处理，将处理结果会返回给Client Agent，返回中会包含Artifact。

- Agent card包含被调用方Agent的一些基本信息，包含名称、描述、技能、url等等
- Message：用于在Agent之间交换信息的一个结构体，会包含Client Agent希望Server Agent解决的问题。
- Artifact：表示被调用Agent的产物，算是返回结果中比较重要的一个部分，包含问题的答案

A2A协议是支持流式返回的，Agent card中有个字段就是用来标记被调用Agent是否支持流式返回，如果支持的话，那么在后续的请求中的method字段会是"message/stream"说明希望被调用的Agent进

行流式返回，后续被调用Agent给出的返回会包含多个部分，首先是更新任务状态为已提交，然后后面每个部分都包含着一部分的流式回答，并且有个字段标记是否是last-chunk，变成true的时候就表明流式返回全部结束，只需要拼接起来就可以得到完整的答案，最后是更新任务状态为完成，至此所有返回内容都结束了。

15. 什么是流式返回？

正常的请求是发送一次请求然后返回一次响应，而流式返回就是发送一次请求然后返回多次，每次响应一部分数据，用户可以及时看到模型的返回，出来几个字就看到几个字，等全部返回完之后会关闭连接。

16. openclaw具体实现分析，如何从零实现一个类似openclaw的agent

内置工具：

- 命令行工具：执行命令行操作，可以使用langchain中内置的一个ShellTool包
- python代码解释器工具：赋予python代码的运行环境，也可以使用langchain中内置的pythonREPLTool包实现
- fetch网络查询工具：获取指定网络url的资源，也可以使用langchain中内置的RequestGetTool
- 文件读取工具：读取指定文件的内容，同样也可以使用langchain中内置的FileReadTool
- RAG检索工具：当用户需要查询指定知识库的时候，Agent可以调用此工具进行深度检索。也可以通过langchain实现，支持扫描指定目录下的文档，构建本地索引，实现混合检索（关键词BM25检索+向量检索），索引文件需要持久化。

AgentSkills：

有了上述的工具支撑，其实Skills实现起来就比较简单了。第一个流程是载入流程，就是能够读取指定skill文件夹下所有skill的元数据信息，并且要校验必须包含Skill.md，利用文件读取工具读取元数据并添加到系统提示词里。第二个就是调用部分，载入后大模型已经能够知道所有的Skills技能，然后就可以通过function calling的方式在遇到相关问题的时候去调用对应Skill.md的内容，后续即可按照Skill.md去执行后续流程。

记忆系统：

Openclaw分为短期记忆，长期记忆和日常记录三个部分，短期记忆是当前会话的上下文，会话结束后默认清除并且受限于上下文窗口，长期记忆指的是Memory.md里存放的内容，其中会记录一些跨会话的重要内容，具体会由AI决定何时触发写入，日常记录是每天的会话内容，比如当天完成的任务，对话中重要的上下文，遇到的问题和解决方案，这是每天会自动生成并更新。

同时记忆的进化和演变也算是一个亮点，记忆演变的意思就是记忆不仅仅是存储下来，还可以对以前的记忆进行一些更新，这样会给人一种智能体越用越智能的感觉，由于记忆系统的不断进化，处理问题的能力也会越来越好，在存储记忆的时候可以总结出一些关键词，然后根据关键词进行检索，对以前一些相关的记忆可以做进一步修改和完善。

17. 如何处理agent处理问题时记忆太多导致混乱的问题？

可以采用工作记忆压缩加外部存储的方式（短期记忆+长期记忆），agent在处理任务的时候会将一个复杂的任务分解成多个子任务，在处理子任务的内部可能也会有很多步骤过程导致工作记忆太长影响agent的决策导致最终任务失败，可以在每个子任务完成的时候就进行一次压缩总结，只记录子任务的目标和最终完成的结果，具体的流程这些信息可以保存在外部存储中，这样agent的工作记忆会变得很简洁，如果在后续子任务中需要参考前面某个子任务的流程也可以去对应的外部存储中检索到，并不会导致记忆丢失的问题。

在压缩方面也有很多可以优化的，比如可以采用多维度摘要的方式，压缩成一段结构化的JSON，其中只保留重要的信息，然后压缩完成后可以让模型进行自我反省一下，现在压缩的内容是否包含了原来的核心内容。对于保留信息的选择，肯定需要保留的就是用户的意图，关键的参数，目前的进展结果，待办与遗留的问题等等，可以丢弃掉的就是一些废话，重复的system prompt，过时的中间过程这些，这样就可以很好处理上下文窗口带来的问题。

18. CLI和MCP工具的差异体现在哪些方面？为什么现在越来越多的Agent选择用CLI？

MCP相比CLI一个最大的缺点就是token的消耗，agent在处理用户请求的时候，不仅仅将用户的问题发送给大模型，还会连带所有工具的信息（工具名，描述，参数列表）全部发送给大模型，这样大模型才知道该调用哪个工具解决，一旦连接的MCP工具过多，就十分消耗token，而CLI只需要传一个bash工具，具体的指令由大模型会自己生成，大部分常见的指令大模型在训练的时候就已经学会了，如果有不常见的或自己写的只需要附带一份说明文档即可，一般会以Agent Skills形式实现，CLI的token消耗量相比下来少很多。

第二点就是在执行效率上，常见的Agent会采用ReAct模式去执行，要经过不断循环的思考-调用工具-观察工具执行结果，而这中间都需要依赖大模型的参与，整体的执行效率自然就低下了，而CLI的工具只是简单的bash程序，在其中可以写很多条串行的命令，整个流程只需要一步思考-执行bash程序-观察执行结果，bash程序一次执行多条命令行，执行效率来说高很多。可能会觉得那把多个MCP工具也缝合到一起形成一个新的工具不也可以达到同样的效果吗，理论上来说是的，但是一旦需求改变，那又得需要重新开发新的工具，每次都重新开发也很复杂，而CLI工具只需要在原来基础上增加几条命令就可以了，这是CLI工具扩展性的天然优势。

但是MCP也有着自己的独特优势，就是更可控、更安全，CLI工具一次生成多条命令，生成的命令不一定是完全正确的，一旦生成错了整个执行流程就会报错，相比MCP工具不依赖于大模型生成命令，更加可控，并且大模型生成的命令不一定是安全的，万一生成了一些rm -rf就会导致误删一些文件，这是非常危险的行为，简而言之就是CLI工具的执行效果依赖于大模型生成命令的正确与否，作为人来说心里会没底。

19. Harness Engineering是什么？

如果把大模型比作一匹野马，那harness就是马具，通过harness来控制大模型在一个安全可控的环境下运行，其实许多团队研究都发现，现在的瓶颈根本不在单个模型的能力，而是在于基础设施，同样的模型，不同的harness最终的效果是不同的，现在的agent会有一些问题，比如在完成一个复杂长任务下，容易一步到位，在一个会话中完成所有的内容，这样的后果就是上下文窗口耗尽，留下一堆半成品，下一个会话启动的时候只能花大量的时间猜测之前干了些什么，还有第二个问题就是当Agent完成一部分功能时候，环顾四周，看到有进展就觉得自己已经全部完成了，提前宣布胜利，这些问题的出现让人们意识到harness的重要性。我觉得harness要解决的有几个重要的内容，第一是约束，限制

agent能做什么不能做什么，第二是环境，也是agent的上下文，告诉agent他应该做什么，这个不能仅仅依赖于上下文窗口，因为不同会话的上下文窗口是不共享的，更需要依赖于外部文件存储，将任务的状态写在外部文档中，第三是验证，需要验证Agent是否正确完成了工作而不是做到一半就停了，这个可以采用另一个agent作为评审，为每次agent完成的任务进行评估从而督促其正确完成任务，第四是纠正，当agent执行出错的时候，需要及时修复，完善反馈循环和自我修复机制，不能在错误的路上越走越偏导致最终任务失败。

20. Claude Code源码分析

整体架构：首先是执行Agent Loop前的准备工作，需要获取当前项目的环境，加载Claude.md和memory目录下的记忆文件作为系统上下文，以及工具的注册操作，将内置工具和MCP工具都进行注册，后续才进行Agent Loop，结束之后需要调用终止工具跳出循环，终止工具包含两个，一个是大型模型认为做完了终止，另一个是大型模型可能需要用户参与比如需要额外需求时也会终止，终止之后对话结束，触发Auto Memory记录重要信息。整体的Agent Loop依然采用经典的ReAct模式设计的，每次进行调用LLM，执行工具，返回结果的循环，直到最终生成符合用户的问题的输出，通过一个state状态机来控制每一轮的迭代，state中变量包含对话历史，工具上下文，自动压缩跟踪，是否尝试压缩，迭代轮数，重试次数等信息，每轮迭代后才会对state进行完整替换更新，避免跨迭代的意外修改。在循环开始时先获取当前迭代的state状态，然后进行skill discovery过程，再获取compact边界之后的历史对话消息（节省token），后续进行调用LLM，执行工具，获取结果，判断是否退出，再将结果添加到消息进行下一轮迭代。在循环中还会持续追踪当前token的消耗数，当token budget耗尽时会终止循环。

工具调用：所有的工具都是实现了自定义的Tool接口，里面包含一个核心的执行函数以及一些变量，包括是否可并行、是否可逆，用户中断怎么办等。所有工具通过三个函数组装成工具池，分别是返回所有内置工具，按照权限和上下文过滤工具，合并所有内置工具+MCP外部工具并重排序，内置工具包含文件操作、shell执行、搜索、agent协作、任务管理等40多个工具，在每个工具执行前都需要进行权限检查链，先进行输入参数检查，再进行权限检查，先进行deny规则匹配，匹配到的话直接拒绝，在进行ask规则匹配，匹配到弹出对话框让用户确认，最后才可以通过，比如如果是只读的允许自动执行，如果是rm删除必须询问，检查通过后才可以进行工具实际调用，工具调用前后也都会设置一些hooks，这个有点像langchain中有个middleware中间件的组件，可以实现一些必要的功能，就比如在工具调用前实现权限校验的操作，在工具调用后执行压缩相关操作。每个工具都有最大结果上限，如果超出上限（如读取超大文件）会将结果写入磁盘临时文件，只保留包含文件路径的摘要，防止单个工具结果撑爆上下文窗口。

并发执行：流式执行引擎是整个工具调用系统中精妙的一个部分，在LLM流式输出的同时，就开始并发执行工具。工具的调用有专门的管理逻辑，会有一个工具执行队列保存所有待执行的工具，如果所有工具全是只读的那就可以立即全部并发执行，如果有写入的工具，必须等所有在执行的工具完成才能独占执行，非并发工具必须串行执行。错误传播机制也很重要，当一个工具执行出错的时候，会立即取消其所有并发执行的兄弟工具进程，避免无效工作继续消耗资源。只是取消当前批次的工具，并不会影响整个流程生命周期。最终结果按照最初LLM给出的tool use的顺序接收，而不是工具完成的顺序，保证并发执行下最终工具结果是正确的。

推理模式：cc支持thinking的功能，在模型给出最终答案之前进行内部推理，用于处理复杂问题，这个模式可以由模型自行思考决定是否开启，也可以由用户指定开启或者关闭。推理内容会在当前响应轮次内跨tool use保留，确保多轮工具调用后还可以记住自己的推理的内容。

多agent协作：Task是多agent协作场景下的工作单元，当一个主agent将大任务拆分给多个子agent去执行时，就是通过task来追踪每个子任务的状态。task会包含当前任务的描述，执行此任务的agent id，本任务完成后才能开始的任务列表以及必须先完成才能开启本任务的任务列表。多个agent对于同一个task的读写操作也都是有相关锁机制控制的，就是为了防止多个agent竞争同一个任务。在开始的时候，主agent会创建多个子agent，并且设定好共享的任务列表，主agent可以显示分配也可以让子agent自行认领任务，拿到任务后，每个子agent就可以去独立执行了，拥有各自的上下文窗口，在任务进行过程中，各个子agent之间也可以相互通信，可以是指定某个队友，也可以是broadcast广播，当任务完成后，有依赖关系的后续任务会自动解锁，子agent会主动通知主agent并且自行去认领下一个任务。

对话压缩：当要接近上下文窗口上限的时候就需要对历史会话做压缩。首先是微压缩，在每次api请求调用前都会触发，优化单次的token消耗，又分为两种压缩策略，分别针对不同的情况，一种是缓存还存在的条件下采用**Cached MicroCompact**，如果直接采用传统的方式将旧工具的中间结果删掉，当这段提示词发送给大模型的时候，由于修改了中间内容导致缓存失效，后续所有的就得重新当作一个新的问题来处理 and 计算，所以采取了一种更为聪明的方式，客户端发送给服务端的prompt不变，但是额外标记了需要删除中间旧工具的字段，服务端比对缓存可以命中，并且能够知道需要删除哪部分信息，由服务端去做具体的删除工作。另一种是缓存不在了，也叫做**Time-Based MicroCompact**，这个是由于缓存的TTL大概就是一个小时，如果用户离开对话超过60分钟，那么默认为缓存已经失效了，那就没必要去防止缓存miss了，直接在prompt中删除旧工具结果再发送。第二级叫做

SessionMemory Compact，这个的核心理念是不要等到需要压缩的时候才压缩，而是平时就开始为压缩做准备，在每一轮对话后，都会fork一个子Agent从对话中提取出关键信息，写入文件当中，当上下文真的需要压缩的时候，直接使用这个记忆文件作为摘要，不需要额外调用LLM去实现压缩，实现0延迟压缩，但要注意一些关键的内容不能切断，比如工具调用和工具结果，还有整个thinking推理链。当前两种都失败的时候，**Full Compact**登场，会调用api来生成一段结构化的摘要，这个更像是一份任务交接文档，保证压缩前后大模型可以无缝衔接完成新的任务，在prompt中提前定义好了这个九段格式，结构化的摘要相比自由文本结果更加可控，保证压缩结果的准确性。

记忆系统：短期记忆其实就是Session Memory，这个被Auto Compact复用。长期记忆依赖于Auto Memory和Auto Dream，Auto Memory是Claude Code自动实现的跨会话记忆，会fork子agent去实现记录会话中的有用的信息，最终会生成一份Memory.md的记忆索引文件，Memory.md是整个记忆目录的索引，Claude通过它来追踪各文件中存储的内容。Auto Dream是在多个对话后才会触发的一个机制，回顾所有记忆文件，合并重复、修正过时信息、修剪冗余，这一步不是新增记忆，而是整理和优化现有记忆。长期记忆都是跨会话持久化存储的，会在新会话时作为记忆上下文加载。还有一个Claude.md这个是由开发者可以手动编写的文档，可以存储一些关键信息，有多个级别的，可以是所有项目共享的，也可以是单个项目下的，或者是某个文件下的，会在会话时加载进系统提示词中。

MCP：CC作为一个MCP Host，内部实现了MCP Client，首先需要解析并加载对应MCP Server的配置文件，这样才知道MCP Server在哪里，然后利用SDK实现了一个MCP Client，向MCP Server建立连

接，MCP有多种通信方式，CC同样实现了四种方式，stdio，sse，streamable，websocket，然后就是获取MCP Server上的tool list，以及在真正需要执行工具的时候发送对应的tool/call请求。

Skill: CC的Skill分为两种，一类是内置的Skill，通过硬编码实现，是CC的一些原生技能，还有一类是用户自定义挂载的，那么需要实现一个指定目录扫描器，从而发现用户挂载了哪些Skill，CC具备热重载机制，内部实现了一个文件监听器，当你修改了某个自定义Skill并保存的时候，就会自动捕捉到，在完全不打断Agent Loop的情况下，实现重新加载对应的Skill。并且CC对Skill也做了统一的降维处理成Tool接口的形式，没有单独开发Skill的调用流程，也就是说对于CC来说，不管是Skill还是MCP Tool还是内置Tool，都是一样的，本质上都是实现了Tool接口的，最后CC还设计了一个Skillfy自我进化功能，当你在对话中引导Agent一步步完成具体流程的时候，可以说一句请帮我把这个流程转化为一个Skill，这个时候Skillfy就会触发，回溯读取Session Memory，提取出对应的信息，生成一个Skill文件，进一步提高了Agent的自我反思与经验沉淀能力。

Prompt: CC的提示词是一个高度模块化、运行时动态拼接的提示词，里面会包含系统的约束、环境的感知、项目上下文（Claude.md文件存在时才会加载）、记忆、工具描述、历史会话和用户指令多个部分，每个部分都会有XML的标签包裹，最后动态拼接成一个完整的提示词，使用XML标签的目的是因为模型在预训练的时候会对标签的注意力机制进行了优化，能够提供绝对清晰的边界，防止指令混淆的情况出现，这个也可以从一定程度上防止提示词注入攻击。

权限治理: 首先是在Permission Mode层面确定当前的模式是什么，CC有多种不同的模式，default模式是可以自由读但是一旦涉及操作就需要用户的确认，plan模式是只能够读完全禁止任何编辑，acceptEdits是文件编辑权限放开，但是shell命令执行这些依然需要人工授权，还有auto模式就是可以自由先斩后奏，确定当前的模式之后会进入下一层规则层，提前定义allow、deny、ask的相关规则并注入到上下文中进入主循环。在工具调用前都会根据permission上下文进行权限判断，是放行、拒绝还是询问，放行的话就会进入到下一层沙箱环境层，permission是决定哪些操作可以放行，而沙箱决定能操作的物理边界，让你运行不代表可以随便运行，当需要执行操作的时候会在沙箱环境中进行运行，在沙箱中对可以访问的文件、网络请求这些都会做对应的限制，进而保证宿主机环境的安全性，在沙箱中做的修改最终会映射到宿主机中，本质是通过目录绑定挂载的思路实现的，也就是说沙箱和宿主机的目录在操作系统底层指向的是同一块硬盘空间，所以在沙箱中做的修改实际上是直接写在了宿主机的硬盘上。

Transformer

1. Transformer的输入是什么？为什么需要位置编码？

模型的输入是一个向量，这个向量由两部分相加得到，分别是词向量和位置向量，词向量就是由单词直接embedding得到，用于描述单词的语义信息，位置编码描述的是单词间的位置信息。

在传统的循环神经网络（RNN）中，模型是按顺序逐个处理单词的，因此它天然地知道单词出现的先后顺序。然而，Transformer 为了实现并行计算，会一次性把一句话里的所有单词都丢进模型。如果

没有位置编码，对于模型来说，“我 喜欢 猫”和“猫 喜欢 我”这两句话的输入特征是完全一样的。因此需要位置编码来表示单词在原始句子中的相对位置和绝对位置的信息。

2. 自注意力机制是怎么计算的？作用是什么？

首先将输入X和三个矩阵相乘得到QKV三个矩阵，再将Q和KT相乘，为了防止内积过大还要除一个数并且进行softmax，这个结果矩阵表示的就是单词与单词之间的注意力系数，即其它单词对当前单词的重要性有多少，再将这个结果和V相乘就可以得到最终的结果，每个单词都包含了其它单词提供的语义信息。

3. 什么是多头自注意力机制？

多头自注意力就是多个自注意力模型，输入X会经过多个自注意力机制得到多个输出结果，每个头处理不同维度的信息，最终拼接起来还是会得到和原来输入X一样维度的矩阵，通过针对性地学习不同维度的表示，利用多个头捕捉不同的子特征，使得最终得到的结果更加丰富准确。

4. 编码器架构是什么样的？

整个Encoder由多个Encoder Block组成，每个block包含多头自注意力，Add&Norm，前馈神经网络，Add&Norm组成，Add&Norm指的是将原始输入和经过多头自注意力和前馈神经网络的输出相加再归一化的过程，最终输出一个编码矩阵，和原始输入X大小是一模一样的。

5. 解码器架构是什么样的？

Decoder的输入是同样是一个包含词向量和位置编码的向量，Decoder block的组成是由两个多头自注意力层和前馈神经网络层组成，不同的是，第一个多头自注意力层加上了Mask，其实就是在计算QKV的时候引入了Mask矩阵，使得每一个单词只能看到前面单词提供的信息，是一个单向的过程，不能利用后面出现单词的信息。下一个多头自注意力层用Encoder的输出计算KV，用上一个层的输入计算Q，作为下一个多头自注意力层的输入。经过多个Encoder Block之后会进入一个softmax层，得到的结果表示某一个词下一个单词是什么的概率。

6. Transformer解决了CNN，RNN哪些问题？

CNN只有局部的窗口视野，如果第一个词和最后一个词有很强的联系，CNN无法处理这种情况，缺乏全局记忆，建立不了长距离的上下文联系。

RNN引入了隐藏状态，可以记住前一个单词的信息，使网络有了记忆，但是依然有两个巨大的缺陷。第一就是串行计算，因为依赖前一个单词的信息，必须等待前一个词计算完成才可以去计算下一个词，效率太低。第二就是随着文本长度增加，信息会被严重稀释，算到最后很可能已经忘了第一个词的信息了。

Transformer通过矩阵的并行计算解决效率问题，充分利用GPU，又通过位置编码解决了并行计算时不知道单词先后顺序的问题，通过注意力机制让每个词都能够和任意词建立联系无论距离多远，解决全局视野和长距离依赖衰减问题。

7. 温度和top-p参数是什么？分别有什么作用？

温度是通过影响softmax函数来改变大模型输出的随机性，温度越高，各个词最终计算出来的概率之间差距会被缩小，生成的随机性增强，温度越低，差距会被放大，生成的词更趋于稳定。

top-p是为了剔除掉那些经过softmax后概率非常低的垃圾词汇又叫做长尾词，防止大模型真的输出这些无关的低概率词汇，通过设定top-p的大小，将最终的答案只锁定在概率top-p的词语当中，防止大模型跑偏。

总结来说，这两个参数值越大，输出就更加多样随机性更高，适用于写小说这种头脑风暴的情况，值越小呢输出就更趋于稳定，适用于写代码这种严谨的情况。